MODULE

10

Advanced functions

Closures and decorators are the #1 ”advanced Python” interview topic, and every web framework is built on them.

YOU WILL LEARN

*args, **kwargs, keyword-only	`01_args_and_kwargs.py`
Scope & closures	`02_scope_and_closures.py`
Functions are objects	`03_first_class_functions.py`
Decorators	`04_decorators.py`
functools: cache, partial	`05_functools.py`
Demo: resilient API client	`06_demo_api_client.py`
Recursion	`07_recursion.py`
Interview Q&A	`INTERVIEW.md`

WHY *ARGS AND **KWARGS

Functions that accept any number of arguments

THE PROBLEM

print() takes 1, 2 or 10 values. A logging wrapper must pass **whatever** it receives on to the real function.

You can’t write a parameter for every possible argument.

THE IDEA

***args packs extra positional values into a tuple. **kwargs packs extra named values into a dict.**

TRY IT *ARGS AND **KWARGS

See what the function receives

```python
def show(first, *args, **kwargs):
    print("first :", first)
    print("args  :", args)
    print("kwargs:", kwargs)

show(1, 2, 3, user="vishal", debug=True)

nums = [4, 5]
opts = {"user": "li"}
show(*nums, **opts)       # * and ** UNPACK when calling
```

OUTPUT

```
first : 1
args  : (2, 3)
kwargs: {'user': 'vishal', 'debug': True}
first : 4
args  : (5,)
kwargs: {'user': 'li'}
```

1. Named parameters are filled **first**.
2. Extra positional values go to **args** (a tuple).
3. Extra keyword values go to **kwargs** (a dict).
4. At the call site, * and ** do the opposite: unpack.

TRY IT KEYWORD-ONLY PARAMETERS

Force callers to write the name

```python
def connect(host, *, timeout=5, retries=3):
    return f"{host} timeout={timeout} retries={retries}"

print(connect("db.local", timeout=10))

try:
    connect("db.local", 10)          # which one is 10?!
except TypeError as e:
    print("TypeError:", e)
```

OUTPUT

```
db.local timeout=10 retries=3
TypeError: connect() takes 1 positional
argument but 2 were given
```

1. Everything after a bare * must be passed **by name**.
2. **connect("db", 10, 3)** is unreadable; **timeout=10** is clear.
3. Many library APIs use this, e.g. **sorted(items, key=..., reverse=...)**.

COMMON MISTAKE PARAMETERS

The mutable default argument trap

✗ WRONG

```
def add_tag(tag, tags=[]):
    tags.append(tag)
    return tags

print(add_tag("a"))   # ['a']
print(add_tag("b"))   # ['a', 'b']  ???
```

The default list is created **once**, when Python reads the **def**. Every call shares and grows the same list.

✓ RIGHT

```
def add_tag(tag, tags=None):
    if tags is None:
        tags = []          # a NEW list per call
    tags.append(tag)
    return tags

print(add_tag("b"))   # ['b']
```

Use **None** as the default and create the list inside. This is asked in many interviews.

PICTURE IT FUNCTIONS ARE OBJECTS

A function is a value, like a number

📦 Without the brackets, **len** is just an object you can hand around. With () you **call** it. So you can store functions, pass them in, and return them.

store it

handlers = {"csv": read_csv}

→

pass it

sorted(users, key=get_age)

→

return it

make_multiplier(2) returns a function

This is the foundation for callbacks, closures and decorators.

TRY IT FUNCTIONS ARE OBJECTS

Pick behaviour at runtime

```
def to_upper(s): return s.upper()
def to_title(s): return s.title()

FORMATTERS = {"upper": to_upper, "title": to_title}

style = "title"                          # e.g. from a config file
print(FORMATTERS[style]("hello world"))

users = [("vishal", 31), ("li", 25), ("sam", 40)]
print(sorted(users, key=lambda u: u[1]))
print(max(users, key=lambda u: u[1])[0])
```

OUTPUT

```
Hello World
[('li', 25), ('vishal', 31), ('sam', 40)]
sam
```

1. A **dict of functions** replaces a long if/elif chain.
2. **key=** takes a function: ”sort by what this returns”.
3. **lambda** = a small function without a name, for one-liners only.

WHY CLOSURES

A function that remembers

THE PROBLEM

You want a **counter** for each user: it must keep its number between calls.

A **global variable** is shared by everyone (bug!). A **class** works, but is a lot of code for one number.

THE IDEA

Let a function create another function. The inner function keeps the outer variables alive. That's a closure.

PICTURE IT CLOSURES

Picture it: a backpack

🎒 When an inner function is created, Python packs the outer variables it uses into a **backpack**. The function carries the backpack everywhere, even after the outer function has finished.

`make_counter()`
runs once and creates **count = 0**

→

`returns next_number`
the inner function + its backpack {count}

→

`c1() c1() c1()`
each call opens the SAME backpack: 1, 2, 3

Call make_counter() again and you get a **new** function with a **new** backpack.

TRY IT CLOSURES

Build it and run it

```
def make_counter():
    count = 0                    # lives in the backpack

    def next_number():
        nonlocal count           # change the OUTER count
        count += 1
        return count

    return next_number           # the function, not a call!

c1 = make_counter()
c2 = make_counter()
print(c1(), c1(), c1())
print(c2())
print(c1.__closure__[0].cell_contents)
```

OUTPUT

```
1 2 3
1
3
```

1. **make_counter()** returns a function.
2. c1 and c2 have **separate** backpacks.
3. **__closure__** lets you look inside the backpack.

STEP BY STEP CLOSURES

What happens on each line

LINE	C1'S COUNT	C2'S COUNT	PRINTED
`c1 = make_counter()`	0	-	
`c2 = make_counter()`	0	0	
`c1()`	1	0	1
`c1()`	2	0	2
`c1()`	3	0	3
`c2()`	3	1	1

> Same code, two independent states. That is exactly what a tiny class would give you, without the class.

COMMON MISTAKE CLOSURES

Bug #1: changing an outer variable

✗ WRONG

```
def make_counter():
    count = 0
    def next_number():
        count += 1      # UnboundLocalError!
        return count
    return next_number
```

count += 1 means "assign", so Python treats count as a NEW local variable that has no value yet.

✓ RIGHT

```
def make_counter():
    count = 0
    def next_number():
        nonlocal count  # use the outer one
        count += 1
        return count
    return next_number
```

nonlocal says: "this name belongs to the enclosing function". Reading needs nothing; assigning needs nonlocal.

COMMON MISTAKE CLOSURES

Bug #2: closures in a loop (late binding)

✗ WRONG

```
funcs = [lambda: i for i in range(3)]
print([f() for f in funcs])

# [2, 2, 2]  ???
```

The backpack stores the **variable i**, not its value. When you call the functions, the loop has finished and i is 2.

✓ RIGHT

```
funcs = [lambda i=i: i for i in range(3)]
print([f() for f in funcs])

# [0, 1, 2]
```

A **default argument** is evaluated **now**, so each function gets its own copy of i.

WHY DECORATORS

Add behaviour without touching the code

THE PROBLEM

Your boss asks: **”log how long each of our 40 API functions takes.”**

Copy-pasting timing code into 40 functions is slow, ugly and easy to get wrong.

```
@timer
def get_orders(user_id): ...
```

THE IDEA

Write the timing code once as a wrapper, then put @timer on top of any function.

PICTURE IT DECORATORS

Picture it: gift wrapping

🎁 A decorator **wraps** your function like gift paper. Callers still call the same name, but now they get the **wrapper**, which does something before and after the real function.



@timer above a def is just short for **get_orders = timer(get_orders)**.

TRY IT DECORATORS

Your first real decorator

```python
import functools, time

def timer(func):
    @functools.wraps(func)            # keep name + docstring
    def wrapper(*args, **kwargs):     # accept ANY arguments
        start = time.perf_counter()
        result = func(*args, **kwargs)
        ms = (time.perf_counter() - start) * 1000
        print(f"{func.__name__} took {ms:.0f} ms")
        return result                 # don't forget!
    return wrapper

@timer
def slow_add(a, b):
    time.sleep(0.05)
    return a + b

print(slow_add(2, 3))
```

OUTPUT

```
slow_add took 50 ms
5
```

1. **timer** receives the function.
2. **wrapper** runs before + after it.
3. ***args, ****kwargs** pass every argument through.
4. **return result**, or the caller gets None.

PICTURE IT DECORATORS

Decorators WITH arguments: one more layer

@retry(times=3) first **calls** retry(times=3). That call must return a normal decorator. So you need three functions, like boxes inside boxes.

`retry(times)`
remembers the settings (a closure!)

→

`decorator(func)`
receives the function

→

`wrapper(*args)`
runs on every call, uses times

Reading order: settings → function → call.

TRY IT DECORATORS

@retry: the classic interview exercise

```python
def retry(times):
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            for attempt in range(1, times + 1):
                try:
                    return func(*args, **kwargs)
                except ConnectionError:
                    print(f"attempt {attempt} failed")
            raise ConnectionError("gave up")
        return wrapper
    return decorator

@retry(times=3)
def download():   # fails twice, then works
    ...
```

OUTPUT

```
attempt 1 failed
attempt 2 failed
data.csv downloaded
```

1. Only catch errors that a retry can fix (**network**), never bugs.
2. Real projects add a **delay** that grows: 1 s, 2 s, 4 s.

COMMON MISTAKE DECORATORS

Bug: forgetting functools.wraps

✗ WRONG

```
def timer(func):
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)
    return wrapper

@timer
def get_user(): """Load a user."""

print(get_user.__name__)   # wrapper  ???
```

Every decorated function now looks like **wrapper**: logs, error messages, docs and FastAPI routes get confused.

✓ RIGHT

```
def timer(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        return func(*args, **kwargs)
    return wrapper

print(get_user.__name__)   # get_user
print(get_user.__doc__)    # Load a user.
```

@functools.wraps(func) copies the name, docstring and signature onto the wrapper. Always add it.

IN REAL JOBS DECORATORS

You will meet decorators on day one

FastAPI / Flask: routes

```
@app.get("/orders/{order_id}")
def read_order(order_id: int): ...
```

Django: permissions

```
@login_required
def dashboard(request): ...
```

Caching expensive work

```
@functools.lru_cache(maxsize=1024)
def exchange_rate(currency): ...
```

pytest: test setup

```
@pytest.fixture
def client(): return TestClient(app)
```

TRY IT FUNCTOOLS.LRU_CACHE

Remember results: from 21,891 calls to 21

```python
from functools import lru_cache

calls = 0
def fib(n):
    global calls; calls += 1
    return n if n < 2 else fib(n - 1) + fib(n - 2)
fib(20); print("plain :", calls, "calls")

calls = 0
@lru_cache(maxsize=None)
def fib(n):
    global calls; calls += 1
    return n if n < 2 else fib(n - 1) + fib(n - 2)
fib(20); print("cached:", calls, "calls")
print(fib.cache_info())
```

OUTPUT

```
plain : 21891 calls
cached: 21 calls
CacheInfo(hits=18, misses=21, maxsize=None,
currsize=21)
```

1. **Memoisation**: same arguments, so return the stored result.
2. Arguments must be **hashable** (no lists, no dicts).
3. Never cache data that **changes** (DB rows, prices, time).

TRY IT FUNCTOOLS.PARTIAL

Pre-fill some arguments

```python
from functools import partial

def send(channel, message, urgent=False):
    flag = "!!" if urgent else ""
    return f"[{channel}] {flag}{message}"

alert = partial(send, "ops", urgent=True)
print(alert("disk 95% full"))

to_int = partial(int, base=2)
print(to_int("1010"))
```

OUTPUT

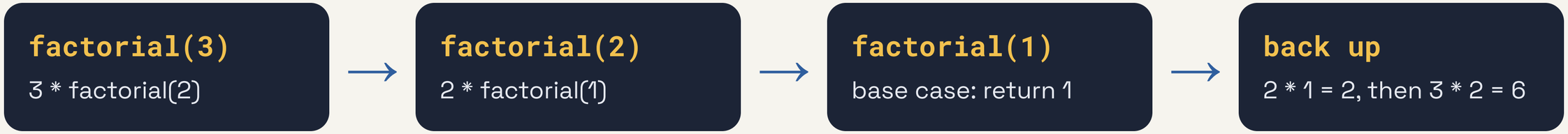
```
[ops] !!disk 95% full
10
```

1. **partial** returns a new function with some arguments fixed.
2. Great for callbacks that must take fewer arguments.
3. Like a closure, but with no code to write.

PICTURE IT RECURSION

A function that calls itself

🪆 Like **Russian dolls**: open a doll, find a smaller one, until the smallest doll can't be opened. That smallest doll is the **base case**.



Each call waits on the **call stack** until the smaller call returns.

TRY IT RECURSION

Walk a nested folder tree

```
tree = {"src": {"app.py": 120, "utils": {"db.py": 80, "auth.py": 60}},
        "README.md": 10}

def total_size(node):
    if isinstance(node, int):        # base case: a file
        return node
    return sum(total_size(child)     # smaller problem
               for child in node.values())

print(total_size(tree), "KB")
```

OUTPUT

```
270 KB
```

1. Recursion fits **nested** data: folders, JSON, menus, comment threads.
2. Every recursive function needs a **base case**.
3. Python stops at about **1,000** levels: **RecursionError**.

INTERVIEW SAY THIS IN THE INTERVIEW

“What is a decorator, and how does it work?”

- A decorator is a function that **takes a function and returns a new function** that adds behaviour.
- **@timer** on top of a def is just **f = timer(f)**.
- The wrapper is a **closure**: it remembers the original function.
- I always use **functools.wraps** to keep the name and docstring.
- I use them for logging, timing, caching, retries and access checks.

```
def timer(func):
    @functools.wraps(func)
    def wrapper(*args, **kwargs):
        start = time.perf_counter()
        try:
            return func(*args, **kwargs)
        finally:
            print(time.perf_counter() - start)
    return wrapper
```

Follow-up they ask next: ”Write one that takes arguments” (3 levels: settings, function, call) and ”In which order do stacked decorators run?” (bottom one wraps first).

INTERVIEW SAY THIS IN THE INTERVIEW

“What is a closure? Give a real example.”

- An inner function that **remembers variables from its enclosing scope**, even after the outer function returned.
- Python stores them in **cells**: you can see them in **func.__closure__**.
- To **change** such a variable you need **nonlocal**.
- Real example: a **dependency factory** in FastAPI, **require_role(”admin”)**, returns a checker that remembers the role.

```
def require_role(role):
    def checker(user=Depends(current_user)):
        if user.role != role:
            raise HTTPException(403)
        return user
    return checker
```

Follow-up they ask next: ”What is the late-binding problem?” (capture with i=i) and ”Closure or class: when do you use which?” (one or two values: closure; more state or methods: class).

RECAP REMEMBER THESE

Module 10 in 6 lines

1. ***args** = tuple of extras, ****kwargs** = dict of extras; never use a mutable default.
2. Functions are objects: store them, pass them (**key=**), return them.
3. A **closure** remembers outer variables; assigning needs **nonlocal**.
4. **@deco** = **f = deco(f)**; always **functools.wraps**; arguments need 3 levels.
5. **lru_cache** for repeated pure calls; **partial** to pre-fill arguments.
6. Recursion needs a **base case**; deep data needs a loop and a stack.

PRACTICE NOW

Do exercises/ex10, then write @log_calls that prints the arguments and the result of any function.